

LTM Parity Specification Draft

A pre-implementation design review for Urban Cybernetics

Status	Draft for architectural review; not an implementation plan
Repository snapshot	main at cb95b572c253be232eb5fc63a911d7c4e0717a8c
Verification baseline	241 tests and 77 subtests passed on 23 June 2026
Prepared from	Yperman thesis, vehicle-tracking LTM paper, literature review, current repository, and prior audit findings
Normative scope	Academic triangular-FD LTM parity while preserving UC's event-led packet architecture

Central design judgement. UC does not need to replace the event log with cumulative curves to reach parity. It does need to prove that the cumulative curves are a complete, lossless projection of canonical events and that every physical transition is admissible under LTM-consistent link and node constraints.

Document purpose

This document specifies the conditions under which Urban Cybernetics may claim academic LTM parity. It records required model semantics, evidence, dependencies, and validation gates before implementation begins. It deliberately avoids algorithms, pseudocode, class designs, and migration plans. Where the literature admits multiple valid formulations, this document defines the narrowest claim compatible with UC's stated research direction.

Preserved architectural decisions

- The packet lifecycle event log remains canonical physical truth.
- Packet identity is preserved and one packet is the base conservation unit.
- Topology remains immutable for the lifetime of a run.
- Observability remains a sampled, delayed artifact layer and never becomes physical truth.
- Routing authorities remain external to loading and cannot mutate physical state.
- The loading engine remains the sole owner and writer of physical state.

Compelling-reason threshold. No finding in the reviewed literature requires abandoning these decisions. The literature constrains the physical equivalence that the event stream and its projections must demonstrate; it does not require cumulative-count arrays to supersede the event log as UC's audit authority.

Evidence and language

Confirmed fact: Directly supported by current source, tests, or a cited paper.

Design judgement: A proposed scope or interpretation required to make the parity claim precise.

Open decision: A scientific choice that must be ratified before parity work can be accepted.

Shall: Required for the parity claim described in this document.

Should: Expected unless a documented alternative demonstrates equivalent validity.

May: Optional or extension behaviour outside the smallest viable parity claim.

1. Parity Claim Definition

1.1 What parity means

For UC, academic LTM parity shall mean that, for the declared model scope and admissible inputs, the event-led packet simulation produces boundary cumulative counts, link demands, link supplies, node transition flows, packet order, travel times, queues, and conservation outcomes that are equivalent to or within a declared discretisation bound of the selected triangular-FD LTM reference.

Parity is a behavioural and evidential claim. It is not a requirement to copy Yperman's internal representation. An implementation that keeps events canonical is parity-eligible only if the LTM state and all physical decisions can be reconstructed deterministically from those events, immutable topology, declared governance inputs, and the fixed run configuration.

1.2 Smallest claim scope

- Deterministic, first-order, triangular-fundamental-diagram link dynamics.
- Fixed admissible timestep with explicit free-flow and backward-wave lags.
- Unit vehicle packets; no weighted or mixed-granularity packets.
- One-to-one nodes, strict-FIFO route-encoded diverges, and declared-priority merges.
- Fixed route intent during the validated loading interval.
- Aggregate and route-disaggregated cumulative boundary counts.
- Canonical synthetic validation and bounded comparison with a continuous or trusted reference LTM.

1.3 Explicit exclusions from the first parity claim

- General concave or non-triangular fundamental diagrams.
- Capacity drop, hysteresis, lane changing, overtaking, and heterogeneous vehicle dynamics.
- General multimovement urban intersection models and multilane conflict resolution.
- Stochastic LTM mechanics.

- Continuous-time or asynchronous event-based LTM variants.
- Weighted packets, mixed conservation units, or mid-link packet splitting.
- Empirical calibration validity for Cork, Anaheim, Los Angeles, or any other city.

1.4 Claim tiers

Tier	Claim	Minimum evidence
T0	Architecture-compatible	Preserved ownership boundaries and event/count consistency only.
T1	Link parity	Single-link cumulative, sending, receiving, travel-time, and timestep evidence.
T2	Minimal network parity	One-to-one, strict-FIFO diverge, declared-priority merge, and spillback evidence.
T3	Packet multi-commodity parity	Packet-to-count ordinal mapping and route-disaggregated cumulative consistency.

Claim boundary. A city-scale run is not a higher parity tier. Scale, calibration, and empirical plausibility are separate evidence dimensions that become meaningful only after the relevant mechanism tier passes.

2. Current Architectural Baseline

At the reviewed commit, the loading engine owns an append-only event log and materialised current views for link storage, membership, entry metadata, queues, completed packets, route indices, and same-tick receiving. Static Link records are immutable. Cumulative counts and historical storage are derived from LINK_ENTRY and LINK_EXIT events. Sending, receiving, merge, diverge, and queue semantics are explicit and extensively tested, but the repository itself describes the kernel as a simplified packetised LTM-style model rather than a full LTM solver.

The current architecture already protects the boundaries most likely to be damaged by a parity programme: topology does not own dynamic state; observations are not truth; authorities consume supplied frames rather than loading internals; derived summaries are read-only; and packet events remain available for conservation and inspection.

Repository evidence: docs/architecture/core_principles.md:4-28; docs/specifications/loading_state.md:7-33, 60-99; docs/architecture/vertical_slice.md:31-54, 179-224.

3. Cumulative Counts

Parity disposition	Required for parity
Gap classification	Foundational; validation-related; performance-related at scale
Current assessment	UC is accounting-compatible but not yet mechanically equivalent to canonical LTM.

Current UC behaviour

UC records LINK_ENTRY and LINK_EXIT as canonical packet events. Public cumulative entry, exit, count-series, and storage views are reconstructed from those events. Current storage and packet membership are also materialised inside the loading engine and checked against the event history. The counts are therefore deterministic projections of canonical truth. They are not presently the complete state used to determine all physical transitions: current sending is selected from packet entry metadata and current receiving is selected from materialised storage and per-tick slot limits.

Academic LTM behaviour

In Yperman's LTM, the upstream and downstream cumulative vehicle functions are the minimum link state. Link volume and link travel time are derived from the vertical and horizontal separations of those curves, and route-disaggregated cumulative counts support route travel times. The vehicle-tracking extension preserves cumulative inflow and outflow while mapping each integer increment to a specific vehicle. Academic parity therefore concerns the completeness and mechanics of the cumulative-count representation, not which data structure is declared canonical.

Gap and parity condition

The event log can remain canonical, but parity requires a lossless, time-indexed cumulative-count projection that is sufficient to reproduce every link demand, link supply, transition flow, storage value, and packet ordinal. Current counts satisfy accounting identities but do not yet establish that physical decisions are LTM functions of lagged boundary counts.

Dependencies

- Stable LINK_ENTRY and LINK_EXIT semantics and deterministic within-tick event ordering.
- A declared inclusive/exclusive time convention for every cumulative-count query.
- Packet ordering and a one-to-one mapping between count increments and packet identities.
- Fundamental-diagram parameters and timestep semantics for lagged count access.

Risks of implementation

- A second materialised representation could drift from the event log unless disagreement is always treated as a run failure.
- Off-by-one tick conventions could change eligibility, travel times, and observation windows.
- Route-disaggregated histories can increase memory pressure sharply on large networks.

- Historical replay and interrupted-run recovery may disagree if count checkpoints omit event-order information.

Risks of not implementing

- UC would remain a packet queue model with LTM-style terminology rather than a defensible LTM implementation.
- Newell-style link and route travel-time claims would remain unsupported.
- Receiving and spillback could not be shown to arise from the same physical accounting as sending.

Existing tests that constrain the design

- **tests/test_cumulative_boundary_counts.py** - counts match events, remain monotone, respect queues, merges, and diverges
- **tests/test_engine_architecture.py** - event/cache disagreement and event sequence violations are detected
- **tests/test_link_storage_views.py** - storage is derived consistently from link entries and exits
- **tests/test_conservation.py** - whole-run conservation is reconciled from canonical events

Additional tests required before claiming parity

- For every valid run, aggregate cumulative counts reconstructed from events equal all maintained count checkpoints at every tick.
- Every unit increase in cumulative inflow and outflow maps to exactly one packet and preserves packet order.
- Route-disaggregated cumulative counts sum exactly to aggregate counts at every boundary and tick.
- Horizontal count-curve matching reproduces per-packet event travel times under FIFO.
- Lagged count queries at free-flow and backward-wave offsets use an explicitly tested boundary convention.
- Replay from a prefix of the canonical event log reconstructs identical counts and live packet ordinals.
- Interrupted and resumed runs do not duplicate or skip cumulative increments.

Repository: loading/cumulative_counts.py:1-138; loading/engine.py:143-223, 276-350, 1093-1267;

docs/specifications/loading_state.md:15-33, 60-99. Literature: Yperman (2007), Sections 3.1-3.2; de Souza et al. (2025), Sections 3.1-3.3.

4. Fundamental Diagram Consistency

Parity disposition	Required for parity
Gap classification	Foundational; validation-related
Current assessment	UC stores relevant metadata but does not yet enforce a single coherent triangular-FD interpretation.

Current UC behaviour

A UC Link may contain length, lane count, free-flow speed, jam density, backward-wave speed, capacity per hour per lane, tick duration, declared sending capacity, declared receiving capacity, free-flow ticks, and packet storage capacity. Free-flow ticks and storage can be derived from subsets of those fields, but explicit values override physical derivation. The parameters are validated independently for positivity, not jointly for triangular-FD consistency. Backward-wave speed is stored but is not used by the loading mechanics. Canonical topology conversion floors capacity into integer packets per tick, imposes a minimum of one, and currently does not multiply the per-lane capacity value by lane count.

Academic LTM behaviour

The triangular fundamental diagram defines free-flow propagation, congested-wave propagation, jam storage, critical density, and capacity as one constraint system. Capacity is not an independent tuning cap if the remaining triangular-FD parameters already determine it. Link length and timestep convert those physical quantities into free-flow and backward-wave lags and per-step transfer limits.

Gap and parity condition

UC permits physically inconsistent combinations and permits declared packet capacities to be unrelated to the physical metadata. The parity specification must distinguish a legacy or synthetic declared-capacity mode from a parity-eligible physical mode. A parity claim requires one declared interpretation of lanes, units, capacity, storage, free-flow lag, backward-wave lag, and integer discretisation.

Dependencies

- Immutable topology metadata and topology hashing.
- Declared packet granularity and vehicle-equivalent counting basis.
- Fixed timestep duration and a stated timestep admissibility condition.
- Unit conventions for length, speed, density, capacity, and packet counts.

Risks of implementation

- Existing synthetic links rely on independent capacities and large legacy storage defaults.
- Parameter reconciliation will change free-flow times, storage, and throughput in prior scenarios.
- Integer per-tick capacity treatment can bias low-flow links or short timesteps.
- Missing or uncertain real-network lane and jam-density data can create false physical precision.

Risks of not implementing

- Sending, receiving, and spillback cannot be jointly interpreted as one first-order traffic model.
- Capacity and storage may encode incompatible traffic states.
- Results may be strongly timestep-dependent for reasons unrelated to network physics.

Existing tests that constrain the design

- **tests/test_traffic_kinematics_foundation.py** - metadata immutability, positive units, storage scaling, and free-flow scaling
- **tests/test_synthetic_ltm_validation_suite.py** - physical storage and free-flow scaling in small networks
- **tests/test_canonical_sioux_falls_topology.py** - topology conversion and source assumptions

Additional tests required before claiming parity

- Parity-eligible links reject or explicitly classify inconsistent triangular-FD parameter sets.
- Capacity, critical density, jam density, and both wave speeds satisfy the selected physical convention within declared tolerance.
- Per-tick capacity reflects lane count and preserves long-horizon hourly capacity without a minimum-one bias.
- Storage and free-flow/backward-wave lags are unit-consistent across at least two timestep durations.
- Equivalent physical links expressed in different source units yield identical canonical parameters and topology hashes.
- Missing physical metadata prevents a parity claim rather than silently invoking the legacy storage default.
- Parameter provenance and any imputation assumptions are present in the reproducibility artifact.

Repository: core/link.py:12-120; topology/canonical.py:65-115; docs/specifications/loading_state.md:124-165. Literature: Yperman (2007), Section 4.2 and Section 4.6; de Souza et al. (2025), Section 3.1.

5. Sending Functions

Parity disposition	Required for parity
Gap classification	Foundational; validation-related; performance-related
Current assessment	UC preserves minimum traversal time, capacity, and FIFO, but current sending is packet-eligibility logic rather than a demonstrated cumulative-count LTM demand.

Current UC behaviour

A packet is eligible to send after its current link-entry tick plus the link's free-flow ticks. Eligible packets are sorted by entry tick, event sequence, and packet ID. The sendable prefix is capped by declared sending capacity per tick; already queued packets are excluded from the public active sending view. Final-link completion consumes the same sending capacity. Queued releases also consume upstream sending capacity.

Academic LTM behaviour

Canonical LTM sending is the maximum downstream boundary increment permitted by cumulative upstream arrivals shifted by the free-flow travel time and by link capacity. The vehicle-tracking model discretises this demand into integer vehicle units while preserving a bounded relationship to continuous cumulative discharge. FIFO identifies which packets correspond to the permitted increment.

Gap and parity condition

Current UC behaviour often resembles the packet-level result of an LTM sending function, but it is not computed or validated as the lagged cumulative-count constraint. Independent integer capacity per tick also lacks the long-horizon capacity carry used by the discrete vehicle paper.

Dependencies

- Cumulative-count parity and packet-to-count ordinal mapping.
- Coherent free-flow lag and capacity from the fundamental diagram.
- Timestep convention and integer capacity discretisation policy.
- FIFO and queue ownership semantics.

Risks of implementation

- Discharge ticks and completion ticks will shift even on simple existing scenarios.
- Fractional capacity preservation can introduce state that must be replayable from canonical events and configuration.
- A count-derived sending view may expose differences between current packet membership order and cumulative ordinal order.
- Performance may regress if historical count access is not bounded independently of total event history.

Risks of not implementing

- UC cannot show equivalence to Newell/Yperman sending mechanics.
- Low per-tick capacities may be systematically biased by integer truncation or minimum-one conversion.
- Travel-time and throughput results remain tied to bespoke eligibility rules.

Existing tests that constrain the design

- **tests/test_ltm_sending_function.py** - free-flow lag, capacity limit, FIFO order, queue exclusion, and final-link completion
- **tests/test_fifo_receiving_constraints.py** - queued releases consume capacity and blocked heads prevent bypass
- **tests/test_route_progression.py** - one boundary per tick and deterministic route progression
- **tests/test_synthetic_ltm_validation_suite.py** - free-flow and sending-bottleneck scenarios

Additional tests required before claiming parity

- Sending increments match an analytical single-link cumulative-count reference under free flow.
- A saturated link reproduces declared long-horizon capacity for non-integer vehicles per tick.
- No packet sends before the lagged cumulative inflow permits its ordinal to leave.
- The selected packets are exactly the FIFO prefix associated with the sending count increment.
- Sending remains correct after queue formation, queue release, and event-log replay.
- Discrete cumulative outflow remains within the declared bound of the continuous reference across multiple timesteps.
- Equivalent physical scenarios at two admissible timestep sizes converge within the declared discretisation tolerance.

Repository: loading/sending.py:22-82; loading/engine.py:487-517, 615-776. Literature: Yperman (2007), Section 4.6; de Souza et al. (2025), Sections 3.1 and 3.3.

6. Receiving Functions

Parity disposition	Required for parity
Gap classification	Foundational; validation-related; performance-related
Current assessment	UC has capacity- and storage-constrained receiving, but not backward-wave or lagged vacancy receiving.

Current UC behaviour

UC receiving availability is the minimum of remaining declared receiving slots and immediate packet storage space, provided engine-owned receiving state is open. Same-tick accepted entries consume both limits. Current storage is entries minus exits, accelerated by an engine-owned materialisation. The backward-wave speed field does not influence receiving.

Academic LTM behaviour

Canonical receiving is a downstream-vacancy constraint derived from cumulative downstream departures shifted by the congested-wave travel time, jam storage, and link capacity. A vehicle leaving the downstream boundary creates vacancy that becomes available at the upstream boundary only after backward propagation. The discrete vehicle model integerises this supply while retaining the same cumulative logic.

Gap and parity condition

UC currently makes vacancy available from immediate storage difference. It therefore captures finite storage and blocking, but not the time required for a vacancy wave to reach the upstream boundary. Receiving closures are external binary gates rather than a substitute for physical vacancy propagation.

Dependencies

- Cumulative downstream counts and explicit backward-wave lag.
- Coherent jam storage and capacity.
- Within-tick ordering of downstream exits and upstream entries.
- Governance semantics for closures or signal gates, kept separate from physical supply.

Risks of implementation

- The timing of queue release and origin admission will change throughout the network.
- Long backward-wave lags require historical count access and increase state retention pressure.
- Same-tick exit/entry ordering can accidentally create premature vacancy.
- Physical supply and governance gating may be conflated if their effects are not separately auditable.

Risks of not implementing

- Spillback remains an immediate storage heuristic rather than kinematic-wave behaviour.
- Congestion onset and recovery may propagate too quickly.

- High-demand urban experiments can understate upstream blocking and queue persistence.

Existing tests that constrain the design

- **tests/test_ltm_receiving_function.py** - open/closed state, slot limits, same-tick acceptance, and transfer gating
- **tests/test_storage_constrained_receiving.py** - finite storage, shared storage, queue release, and upstream retention
- **tests/test_fifo_receiving_constraints.py** - queue head blocking and one-boundary-per-tick semantics
- **tests/test_synthetic_ltm_validation_suite.py** - receiving bottleneck and storage spillback scenarios

Additional tests required before claiming parity

- A downstream exit does not create upstream receiving space before the backward-wave lag has elapsed.
- Receiving increments match an analytical two-link cumulative-count reference.
- Jam storage is never exceeded, including multiple same-tick incoming approaches.
- Long-horizon receiving capacity is unbiased for non-integer vehicles per tick.
- Governance closure and physical receiving shortage produce distinct, auditable causes while yielding valid transition limits.
- Receiving remains correct after replay, interruption, and restoration from a canonical event prefix.
- Timestep refinement preserves queue onset and release timing within declared tolerance.

Repository: loading/receiving.py:27-107; loading/engine.py:519-561. Literature: Yperman (2007), Section 4.6; de Souza et al. (2025), Sections 3.1 and 3.3.

7. Spillback

Parity disposition	Required for parity
Gap classification	Foundational; validation-related
Current assessment	UC has basic blocking-back semantics but not canonical vacancy-wave propagation.

Current UC behaviour

When a downstream link has no receiving slots or storage space, eligible upstream packets do not cross the boundary. They remain physically on the upstream link and may enter a boundary queue. A blocked queue head prevents following packets from bypassing. When immediate downstream space becomes available, the queued FIFO prefix may transfer subject to sending and receiving capacity.

Academic LTM behaviour

In LTM, spillback is not a separate queue rule. It is the network consequence of the receiving function: downstream departures create vacancies; those vacancies propagate upstream at the backward-wave speed; insufficient receiving restricts node transition flows and causes upstream cumulative departures to flatten. Queue formation and dissipation must agree with those curves.

Gap and parity condition

UC enforces conservation and finite storage during blocking, which is an important prerequisite. The remaining difference is temporal and physical: current blocking and release follow immediate storage availability rather than a lagged vacancy process.

Dependencies

- Receiving-function parity.
- Cumulative-count and fundamental-diagram consistency.
- Node transition flows that never exceed receiving.
- Queue semantics that preserve packet identity without becoming a second physical location.

Risks of implementation

- Queue onset, release, and gridlock timing will differ from all existing high-demand runs.
- Cycles in the network can expose deadlock or inconsistent simultaneous-release assumptions.
- Boundary queue events may be mistaken for an additional point queue unless their relation to link storage remains explicit.
- Diagnostics based on queue event duration may change interpretation once physical vacancy delay is introduced.

Risks of not implementing

- Large-network congestion may dissipate too rapidly.
- Upstream bottleneck attribution may be wrong.
- Case studies with sustained demand cannot support first-order traffic-wave claims.

Existing tests that constrain the design

- **tests/test_storage_constrained_receiving.py** - downstream full blocks upstream transfer and queued packets remain upstream
- **tests/test_fifo_receiving_constraints.py** - strict queue-head blocking and conservation during queues
- **tests/test_synthetic_ltm_validation_suite.py** - basic two-link storage spillback
- **tests/test_run_bottleneck_diagnostics.py** - queue persistence is derived from canonical queue events

Additional tests required before claiming parity

- Two-link lane-drop fixture with analytical queue onset, maximum queue, spillback onset, and dissipation times.
- Three-link chain proving that blocking propagates one physical vacancy lag at a time rather than instantaneously.
- Closed-loop network fixture checking conservation and declared gridlock behaviour.
- Spillback never creates a LINK_EXIT before the corresponding downstream receiving increment.
- Queue-event diagnostics agree with cumulative-curve queue estimates under the declared queue definition.
- Timestep sensitivity fixture for spillback onset and recovery.
- Long-horizon high-demand fixture with zero storage violations and zero conservation drift.

Repository: loading/engine.py:662-858; docs/specifications/loading_state.md:116-120, 158-165. Literature: Yperman (2007), Section 4.6; literature review, 'The load-bearing equations' and 'Suggested validation experiments and metrics'.

8. Node Models

Parity disposition	Required for network parity; advanced urban nodes are research-specific
Gap classification	Foundational; validation-related; research-specific
Current assessment	UC has an explicit policy boundary and strong FIFO behaviour, but not an academic first-order node family.

Current UC behaviour

Node records contain immutable connectivity only. The loading engine supplies immutable transfer candidates, downstream slot limits, upstream packet order, and queued-route information to a `NodeTransferPolicy`. The default `GlobalFIFOMergePolicy` inherits strict upstream FIFO and allocates candidates globally by eligibility tick, event sequence, and packet ID. Merges have no declared priority parameters. Diverges follow packet route intent and enforce strict first-vehicle blocking. Signal, turning-capacity, conflict, and invariance semantics are not specified.

Academic LTM behaviour

The link model supplies sending and receiving; the node model determines actual transition flows. At minimum, node flows must conserve vehicles, remain non-negative, satisfy all incoming demand and outgoing supply constraints, preserve route or turning allocation, and have an explicit priority rule when supply is scarce. Yperman treats one-to-one, origin, destination, diverge, merge, and urban cross nodes separately. The discrete vehicle paper preserves FIFO diverges and approximates merge priority shares over time using indivisible vehicle transfers.

Gap and parity condition

The policy interface is a useful ownership boundary, but Global FIFO is a particular fairness rule, not a general merge model. No test establishes unequal priority shares, invariance, demand redistribution when one approach is empty, or signal/conflict semantics. The minimal parity claim should therefore be scoped to one-to-one, strict-FIFO diverge, and declared-priority merge nodes. Signalised and complex cross-node parity should remain a later, explicitly research-specific claim.

Dependencies

- Parity-eligible sending and receiving values.
- Packet route intent and exact upstream ordering.
- Declared node type, priorities, movement feasibility, and deterministic tie-breaking.
- Governance interfaces for signals and closures without topology mutation.

Risks of implementation

- Changing merge allocation will alter queues and route travel times network-wide.
- Priority rules for indivisible packets can create short-horizon oscillation or starvation.
- A general urban node model can become non-unique or under-specified.

- Signal gating can be confused with link receiving unless node and governance effects remain separately attributable.

Risks of not implementing

- Merge outcomes depend on event arrival order rather than declared physical or behavioural priority.
- Anaheim, Cork, and Los Angeles junction results may be structurally wrong even if link physics is correct.
- No credible claim can be made for signalised or priority-controlled urban traffic.

Existing tests that constrain the design

- **tests/test_global_fifo_merge.py** - shared receiving, eligibility ordering, deterministic tie-break, and queued retention
- **tests/test_strict_fifo_diverge.py** - route-intent preservation and blocked-branch FIFO
- **tests/test_fifo_receiving_constraints.py** - blocked queue head cannot be bypassed
- **tests/test_cumulative_boundary_counts.py** - merge and diverge counts remain event-consistent

Additional tests required before claiming parity

- One-to-one transition flow equals the admissible minimum of upstream sending and downstream receiving.
- Unequal-priority merge achieves its declared long-horizon share while using indivisible packets.
- Unused merge share is reassigned without violating demand, supply, conservation, or starvation limits.
- Merge results are invariant to irrelevant link enumeration and dictionary order.
- Strict-FIFO diverge reproduces route-encoded branch counts and blocks following packets when the front branch lacks supply.
- Multi-input, multi-output node fixtures declare whether they are supported, rejected, or research-specific.
- Minimal signal gate fixture validates phase timing, capacity accounting, and conservation without treating the signal as topology.
- All node fixtures report zero negative flows, zero supply exceedance, and zero packet duplication.

Repository: core/node.py:8-14; loading/transfer_policy.py:10-108; loading/engine.py:435-464, 630-870; docs/specifications/node_model_assumptions.md. Literature: Yperman (2007), Chapter 5; de Souza et al. (2025), Sections 3.4-3.5.

9. Multi-Commodity Flow and Packet Identity

Parity disposition	Required for UC packet-LTM parity
Gap classification	Foundational; validation-related; performance-related
Current assessment	Packet identity and route order are among UC's closest alignments with the vehicle-tracking LTM, but route-disaggregated cumulative consistency is not yet established.

Current UC behaviour

Each packet has a stable ID and a fixed ordered route intent. The loading engine records realised LINK_ENTRY events, maintains a route index, and rejects event/cache disagreement. Diverges select the packet's next route link rather than a proportional commodity split. Strict FIFO prevents a later packet from bypassing a front packet bound for a blocked branch. Demand quantities are expanded into unit loading requests before packet admission. Weighted packets and active rerouting are not implemented in the current kernel.

Academic LTM behaviour

Yperman's multi-commodity LTM stores route-disaggregated cumulative counts whose sum is the aggregate count and uses FIFO to derive route travel times. The vehicle-tracking extension associates each integer cumulative increment with a specific vehicle. Its diverge model processes vehicles in entry order and uses each vehicle's next link instead of requiring an exogenous turning fraction. This avoids naive proportional commodity splitting that can violate FIFO.

Gap and parity condition

UC already preserves the identity and order needed by the discrete extension. Parity still requires formal equivalence between packet order and cumulative ordinals, route-disaggregated counts, route travel-time derivation, and node allocations. The base parity claim should remain unit-packet and fixed-route. Weighted packets, mixed conservation units, and mid-route intent changes require separate specifications and must not be folded silently into the parity baseline.

Dependencies

- Cumulative-count parity and exact packet-to-increment mapping.
- FIFO link and node semantics.
- Node models that consume packet routes without violating aggregate sending or receiving.
- Demand and route provenance with deterministic packet admission order.

Risks of implementation

- Route-disaggregated state may dominate memory on networks with many paths.
- Packet order, commodity order, and event order can diverge under rerouting or cancellation.
- Changing packet granularity would alter capacities, storage, FIFO, and all validation tolerances.

- Per-vehicle node processing can become demand-sensitive even where aggregate LTM cost is link-time sensitive.

Risks of not implementing

- UC cannot claim the vehicle-tracking LTM bridge despite retaining packet IDs.
- Route travel times and routing-authority outcomes may not correspond to aggregate physical flows.
- Commodity-specific FIFO violations may remain hidden behind correct aggregate counts.

Existing tests that constrain the design

- **tests/test_packet_identity.py** - packet ID remains stable across lifecycle events
- **tests/test_route_progression.py** - realised link order follows route intent and route-index cache
- **tests/test_strict_fifo_diverge.py** - route-specific branch choice and no overtaking
- **tests/test_demand_manifest.py** - pre-packet demand expansion and deterministic schedules
- **tests/test_engine_architecture.py** - event-derived route position and packet membership consistency

Additional tests required before claiming parity

- Every route-disaggregated cumulative series sums exactly to the aggregate link series.
- The nth cumulative outflow increment maps to the nth admissible packet under FIFO.
- Per-packet event travel time equals the route-disaggregated cumulative-curve travel time.
- Diverge branch counts match route-encoded packet counts without proportional commodity splitting.
- High-commodity fixture checks correctness and memory growth as route count increases.
- Cancellation and run termination preserve count and conservation semantics without ordinal reuse.
- Weighted or cohort packets are rejected from parity-eligible configurations until separately specified.
- Any future reroute fixture preserves prior route history and declares how cumulative commodity identity changes.

Repository: core/packet.py:10-17; loading/engine.py:581-593, 944-1091; demand/scheduled_loading.py:17-120; docs/specifications/packet_semantics.md:19-37, 84-102. Literature: Yperman (2007), Sections 3.1-3.2; de Souza et al. (2025), Sections 3.2, 3.4.2, and 4.2.

10. Validation and Parity Claims

Parity disposition	Required before any parity claim
Gap classification	Validation-related; performance-related; research-specific at city scale
Current assessment	UC has a substantial invariant suite, but it does not yet compare the kernel numerically with a canonical continuous or trusted high-resolution LTM reference.

Current UC behaviour

The current suite passes 241 tests and 77 subtests at repository commit `cb95b572c253be232eb5fc63a911d7c4e0717a8c`. Tests cover event ownership, packet identity, conservation, cumulative counts, free-flow delay, sending and receiving caps, finite storage, basic spillback, strict FIFO, merges, diverges, route progression, observation isolation, authority isolation, provenance, and Sioux Falls loading smoke behaviour. Synthetic validation asserts qualitative and exact packet-event outcomes under today's rules. Sioux Falls smoke runs measure completion, runtime, memory, and event counts but are explicitly not dynamic-model validation.

Academic LTM behaviour

Parity validation must compare cumulative curves, flows, travel times, queue timing, branch shares, and merge priorities against analytically known behaviour or a trusted canonical reference. The vehicle-tracking paper compares discrete and continuous LTM for lane drop, diverge, merge, signal, field calibration, and large-network runtime, and states a bounded cumulative-flow discretisation difference under its assumptions. Validation must separate correctness, discretisation error, parameter transferability, scalability, and empirical calibration.

Gap and parity condition

UC's tests strongly constrain its current semantics, but they mostly prove internal consistency and the intended queue model. They do not establish Newell/Yperman equivalence, bounded error to a continuous reference, timestep convergence, backward-wave timing, priority merge shares, or urban signal behaviour. A passing city run cannot substitute for these fixtures.

Dependencies

- A frozen parity claim scope and reference variant.
- All foundational link, receiving, spillback, node, and commodity contracts.
- Reference inputs, expected outputs, tolerances, and provenance.
- Deterministic run artifacts and failure reporting for partial runs.

Risks of implementation

- A reference harness can accidentally encode the same assumptions as UC and provide circular validation.

- Exact packet-level equality may be inappropriate where the discrete model guarantees only bounded aggregate error.
- Performance optimisations can pass small fixtures while changing long-horizon capacity or queue timing.
- Empirical fit can obscure mechanism errors if calibration and model validation are not separated.

Risks of not implementing

- Academic parity would be a naming claim rather than an evidenced result.
- Cork, Anaheim, and Los Angeles outputs could appear plausible while violating the intended mechanics.
- Previous experiments could be compared across incompatible physical kernels without clear version boundaries.

Existing tests that constrain the design

- **tests/test_synthetic_ltm_validation_suite.py** - current packet-level free-flow, bottleneck, merge, diverge, and spillback scenarios
- **tests/test_traffic_kinematics_foundation.py** - static physical metadata foundations
- **tests/test_sioux_falls_benchmark_loading.py** - non-trivial topology and demand smoke plumbing
- **tests/test_run_provenance.py** - frozen run metadata, configuration, and artifact references
- **tests/test_event_memory_hardening.py** - compact event/packet records preserve semantics

Additional tests required before claiming parity

- Single-link free-flow and capacity-discharge curves against an analytical reference.
- Two-link lane-drop and spillback curves with queue onset, propagation, and dissipation tolerances.
- Strict-FIFO diverge against route-encoded continuous aggregate flows.
- Unequal-priority merge against declared long-horizon priority shares.
- Discrete-versus-continuous cumulative-flow error bound over multiple timesteps and demand profiles.
- Timestep convergence and admissibility suite for free-flow and backward-wave lags.
- Simple signalised interrupted-flow fixture, clearly separated from core network parity if excluded from the first claim.
- Partial-run, interruption, replay, and failed-run artifact validation.
- Scale ladder: synthetic, Anaheim subnetwork, full Anaheim, then Cork/Los Angeles, with runtime and memory budgets.
- Independent reference implementation or archived reference output whose provenance is separate from UC.

Repository: tests listed above; benchmarks/sioux_falls_demand_smoke.py:46-263; docs/policies/benchmark_policy.md:13-59.

Literature: de Souza et al. (2025), Section 4 and Section 5; literature review, 'Suggested validation experiments and metrics'.

11. Cross-Cutting Design Decisions Requiring Ratification

Reference variant: The first claim should target deterministic, discrete-time, triangular-FD LTM with the vehicle-tracking paper as the packet-level compatibility reference.

Canonical truth relationship: The event log remains authoritative. A count disagreement is a parity failure, not a reason to let counts silently override events or events silently overwrite counts.

Packet granularity: Parity-eligible runs use one packet per vehicle-equivalent unit. Weighted packets remain outside the baseline until FIFO, storage, count, and node semantics are separately specified.

Timestep: The parity configuration declares an admissible timestep relative to free-flow and backward-wave travel across every link. A configuration outside that domain cannot inherit the parity claim.

Node scope: One-to-one, strict-FIFO diverge, and declared-priority merge are required. Signalised nodes are a separately labelled extension unless included in the claim and validation set.

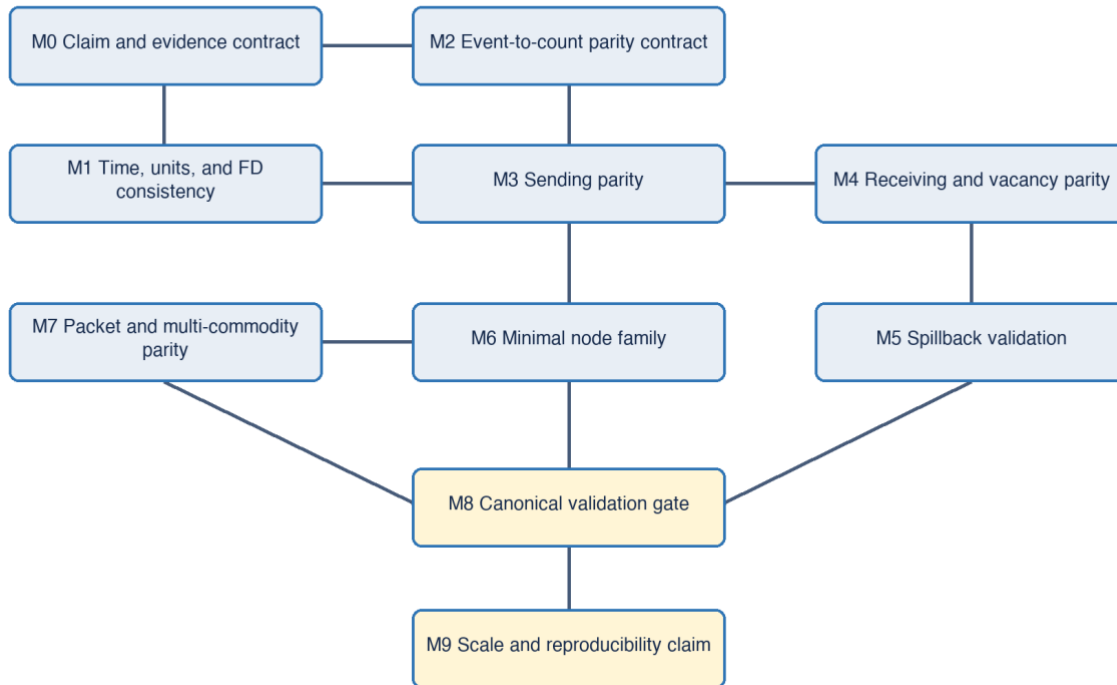
Rerouting: The first parity claim assumes fixed route intent over the validated loading interval. Future rerouting must preserve event history and explicitly define commodity identity changes.

Parity tolerance: Packet-level conservation and identity requirements are exact. Continuous-versus-discrete flow, travel-time, and queue comparisons use declared bounded tolerances tied to timestep and unit size.

Failure semantics: Partial, interrupted, timed-out, or inconsistent runs must be marked non-parity evidence even if their derived summaries can be computed.

12. Dependency Graph of Remaining Milestones

Remaining milestone dependency graph



Arrows indicate prerequisites, not implementation ownership.

Figure 1. Milestone dependencies for the smallest defensible parity claim.

ID	Milestone	Exit condition
M0	Claim and evidence contract	Freeze scope, tolerances, exclusions, source authority, and failure semantics.
M1	Time, units, and FD consistency	Establish parity-eligible physical parameter and timestep interpretation.
M2	Event-to-count parity contract	Prove counts and packet ordinals are complete projections of canonical events.
M3	Sending parity	Validate lagged cumulative demand and bounded integer discharge.
M4	Receiving and vacancy parity	Validate lagged vacancy supply and capacity.
M5	Spillback validation	Validate queue propagation and dissipation across links.
M6	Minimal node family	Validate one-to-one, strict FIFO diverge, and declared-priority merge.
M7	Packet and multi-commodity parity	Validate route-disaggregated counts, packet mapping, and route travel times.
M8	Canonical validation gate	Independent analytical/reference fixtures and replay/failure evidence.
M9	Scale and reproducibility claim	Staged benchmark evidence after mechanism parity passes.

13. Smallest Viable Path to Parity

1. Ratify the parity claim, reference variant, exclusions, exact-versus-toleranced assertions, and failed-run semantics.
2. Establish one coherent physical interpretation for topology units, triangular-FD parameters, packet granularity, and timestep admissibility.
3. Establish the event-to-cumulative-count and cumulative-ordinal-to-packet contracts, including replay and route disaggregation.
4. Pass link-level analytical validation for sending, receiving, travel time, long-horizon capacity, and backward-wave vacancy.
5. Pass network fixtures for spillback, one-to-one transfer, strict-FIFO diverge, and unequal-priority merge.
6. Pass discrete-versus-continuous or independently archived reference comparisons with declared error bounds.
7. Only then attach the T1-T3 parity claim to a versioned model configuration and begin staged scale validation.

Smallest viable parity. The minimum defensible endpoint is T3 packet multi-commodity parity on triangular-FD links with one-to-one, strict-FIFO diverge, and declared-priority merge nodes. General signals, complex urban cross nodes, non-triangular diagrams, stochasticity, and weighted packets are not prerequisites if the claim excludes them explicitly.

14. Changes Most Likely to Invalidate Previous Experimental Results

The following changes would alter canonical physical event timing or packet movement. Quantitative outputs from prior experiments should not be compared across the boundary without rerunning them under a newly versioned physical-kernel configuration.

Risk	Change	Why previous results move
Very high	Receiving and backward-wave vacancy	Changes queue formation, spillback, release, completion, observations, and authority decisions.
Very high	Fundamental-diagram and storage consistency	Changes free-flow lag, capacity, storage, and topology interpretation.
Very high	Capacity discretisation	Changes long-horizon throughput and the exact tick of packet transfers.
High	Sending derived from lagged cumulative counts	Changes discharge and completion timing.
High	Merge priority model	Changes which approach moves, queue allocation, and route travel times.
High	Timestep admissibility or phase ordering	Changes simultaneous transfers, observations, and causal ordering.
High	Packet-to-cumulative ordinal semantics	May change FIFO outcomes at diverges and per-packet travel times.
High	Origin receiving and demand admission	Changes packet instantiation counts and departure timing.
Moderate to high	Route-disaggregated commodity treatment	Changes branch flows and authority outcome attribution.
Moderate	Travel-time derivation	May leave movement unchanged but changes reported metrics and policy inputs.

14.1 Interpretation for existing UC experiments

- The architectural result that authorities can be isolated from physical truth remains valid.
- The qualitative demonstration that receipt delay can alter visible information remains valid as a system capability.
- Specific route choices, completion ticks, queue durations, throughput, and delay-sensitivity metrics are likely to change after foundational parity work.
- Sioux Falls smoke evidence remains useful as a plumbing and performance baseline, but its physical outcomes are not parity evidence.
- Read-only inspection and provenance schemas remain conceptually reusable, although derived metric values must be regenerated.

14.2 Changes unlikely to invalidate physical results if semantics remain identical

- Replacing event scans with checked materialised indexes that reproduce identical event-led decisions.
- Persisting existing canonical events and run metadata without changing event content or order.
- Adding read-only diagnostics, reports, or validation summaries.
- Adding benchmark adapters that do not alter canonical topology interpretation.
- Optimising immutable record memory layouts while preserving equality, ordering, and full history.

15. Acceptance Summary

Safe to preserve. Event-led truth, packet identity, immutable topology, loading ownership, observation separation, authority isolation, and read-only inspection are compatible with academic LTM parity.

Not safe to claim yet. The current repository does not yet support a claim of triangular-FD LTM parity, backward-wave spillback parity, priority-node parity, or vehicle-tracking LTM equivalence. Existing tests lock important prerequisites and current semantics, not the missing academic equivalence.

Appendix A. Evidence Index

Key	Source	Primary use in this specification
Y07	Isaak Yperman, The Link Transmission Model for Dynamic Network Loading, 2007.	Sections 3.1-3.2, 4.2, 4.6, and Chapter 5.
DS25	Felipe de Souza, Omer Verbas, Joshua Auld, Chris M.J. Tampere, A Mesoscopic Link-Transmission-Model Able to Track Individual Vehicles, preprint dated 4 June 2025.	Sections 3.1-3.5, 4, and 5.
LR	Literature Review for an Academic Link Transmission Model in Urban Cybernetics.	Executive summary; Canonical foundations; Load-bearing equations; Node models, FIFO, and multi-commodity issues; Recommended technical tasks and validation.
UC-CP	docs/architecture/core_principles.md	Ownership, topology, observation, packet conservation, authority, derived-state, and reproducibility invariants.
UC-LS	docs/specifications/loading_state.md	Event log authority, count projections, materialised views, sending/receiving claims, kinematic metadata, and explicit non-claims.
UC-TS	docs/specifications/timestep_semantics.md	Fixed clock, phase ordering, one-boundary-per-tick intent, determinism, and replay.
UC-PS	docs/specifications/packet_semantics.md	Packet identity, granularity, conservation, route intent, and open weighted-packet/rerouting questions.
UC-INV	docs/specifications/invariants.md	I1-I15 invariant statements.
UC-ENG	src/urban_cybernetics/loading/engine.py	Current physical state transitions and event/cache consistency.
UC-LINK	src/urban_cybernetics/core/link.py and topology/canonical.py	Current immutable physical metadata and topology conversion.
UC-TEST	tests/	Current executable behaviour baseline: 241 tests and 77 subtests passed.

Appendix B. Audit Notes

- The literature review's tentative UC capability map was not treated as repository evidence because the review explicitly states that it was not based on a fresh full code audit.
- Current-behaviour statements were revalidated against source and tests at the repository commit recorded on the title page.
- No source code or existing repository documentation was modified for this specification.
- The test suite was executed with Python bytecode writing and pytest cache generation disabled.
- The vehicle-tracking paper was obtained from the cited OSTI primary-source copy and used independently of the literature review summary.